

The Review Mediator

Role-based access and component architecture for an online dispute resolution platform

Role	Frontend Developer (Authentication & Design Systems), Team Alpha
Timeline	Fall 2025, 13 weeks
Team	6 developers, cross-functional
Stack	React, TypeScript, Tailwind CSS, Clerk, Express.js, Supabase (PostgreSQL)

This project was built for an external company sponsor as part of a university capstone. Per the project's intellectual property agreement, I can describe my role, the architecture, and my technical decisions in detail, but I cannot reproduce screenshots of the live product or actual source code. Figures referenced throughout are illustrative diagrams recreating the underlying concepts, not literal implementation artifacts.

The Problem

The Review Mediator (TRM) is an online dispute-resolution platform that provides small businesses one place to see their reviews across platforms, flag the ones that need a response, and manage the resulting disputes end to end.

Small businesses don't have much control over their own reputation on platforms they don't own. A single unfair or policy-violating review on Google or Yelp can sit there for months, and the tools you'd normally use to dispute it are scattered and slow, built around the platform's terms of service instead of how a business owner actually works.

TRM tries to fix the workflow itself, not just the review, letting a business reach out to a reviewer directly without leaving the dashboard, solving the problem of working across a variety of scattered platform tools. My team of six was asked to build the MVP: a working prototype that proved the core loop end to end, covering secure login, role-based dashboards, review import, dispute submission, and case tracking.

Goals & Constraints

Two things shaped every decision that we made: a 13-week timeline, and the fact that we weren't starting from a blank slate.

The inherited code worked for the scenarios it had been written for, but it didn't generalize well. Much of the logic was page-specific, and the architecture had evolved feature by feature rather than around reusable patterns. The bigger issue was access control: login and account creation worked, but once someone was in, they could see everything, with no real distinction between an anonymous visitor, a business user, and an admin.

The team itself was six developers, cross-functional, mentored by a senior developer. Early on we rotated through frontend, backend, and database before settling into fixed roles, so everyone had at least some grounding in the other layers before specializing.

My Role

I worked on the frontend team, mostly on two things that ended up mattering architecturally: authentication and access integration, and design system governance, plus full ownership of the Contact Page feature.

Authentication integration

I implemented the Clerk authentication flow on the frontend, making sure session tokens correctly gated access to protected pages and got attached to outgoing API requests.

Full ownership of two features

I built the Contact Page from front to back, form, validation, Express route, Supabase write, and the role-based dashboard routing logic described below. Most features on the project were split across teams: one team built the interface, another wired up the endpoints, a third handled the database, so owning a feature end to end was rare. The Contact Page became one of the few features where I owned the full stack lifecycle: interface, validation, API integration, and database persistence.

Key Decisions

The Shared Design System

Given the state of the inherited code, we decided to tear most of it down and rebuild using atomic design principles¹ instead of patching around what was already there. A senior developer mentored the rebuild, which helped the team figure out what was actually worth keeping. The taxonomy itself mattered less than what got built on top of it. Using that component architecture, I broke shared UI elements (buttons, form fields, layout shells, and navigation) into reusable components instead of letting each page rebuild its own version of the same thing. But the part of this that actually mattered day to day was the Tailwind CSS style guide I built and maintained on top of those components: a shared set of spacing, type, and color decisions that the rest of the team pulled from instead of picking their own values page by page.

Five other people were building pages at the same time I was. Without a shared system, that's five slightly different interpretations of “the same” button, five different gaps between a label and an input, five different shades of the same gray. The style guide was the thing that kept that from happening, it meant a design decision only had to get made once, in one place, instead of re-litigated on every page it showed up on. That's a small thing to say and a genuinely tedious thing to maintain in practice, since it meant reviewing other people's UI work against a standard I'd set, not just building my own.

Routing by Role, Not by Page

I implemented role-based routing at the router level rather than page by page, because the enforcement point scales with the size of the application instead of with the number of pages: as the surface area grows, the check doesn't move, it just keeps working. The idea came out of a team discussion. Something I said pointed the conversation in that direction, and our senior developer mentor took it from there and fleshed out the actual pattern, drawing on standard role-based access control practice.² I'm the one who implemented it: a check using the user's Clerk session token to decide what a person could see before any individual page even loaded.

- No valid session, and you're routed to the public landing page no matter what URL you typed, so you can't get around the gate by guessing a dashboard URL directly

- Valid session, and you're routed into the authenticated shell, where the dashboard, cases page, and account tools live

Putting the check at the routing layer meant every new authenticated page inherited the correct behavior instead of requiring individual guards. It also created a single enforcement point for debugging and future expansion.

The roles weren't just about who could see what, they were about what each type of user actually needed from the interface. We identified three distinct user journeys, each with different interface priorities:

- An anonymous visitor's job is to become a user, so the landing page leads with a hook toward signing up rather than functionality they can't access yet.
- A business user's dashboard needed to lead with itself: organized, not cluttered, surfacing the full range of functionality without burying any of it.
- An admin's experience deprioritized polish for usability, since admins would be trained to use the tool rather than encountering it cold.

See: Figure D, role-based interface structure

Lessons Learned

This wasn't a solo project, and a big part of what I actually learned was less about React and more about working inside a shared codebase for the first time. Early on I struggled with git in a real team setting: branch hygiene, rebasing onto the latest release branch, and cleaning up a branch before it was ready for review. None of that was intuitive to me at first, and I leaned on our technical advisor and a lot of outside reading to learn collaborative version-control workflows properly. By the second half of the project it had become routine instead of a source of dread.

The bigger shift for me was architectural, not procedural. I already knew component libraries and reusable patterns existed before this project, I just hadn't felt the actual cost of skipping them until I was the person responsible for keeping five other people's pages looking like they belonged together. Even once I understood the theory, applying it wasn't obvious. I remember being genuinely unsure why some organisms in our structure skipped straight to atoms while others routed through molecules first, and why some pages resolved down to a template and others didn't. Atomic design stopped being a term from a lecture slide and became something I had to actually reason through case by case, which is a very different kind of understanding.

Outcome

Going in, the sponsor's expectations were pretty modest, calibrated to what the previous student team had managed: a skeleton frontend that worked but wasn't fully fleshed out, a functional backend with proper endpoints, and a database structured with room to grow.

The frontend portion of the project progressed furthest beyond that initial baseline. We moved past a bare skeleton into a fully fleshed-out UI, building out the Contact Page, the dashboard and menu pages, and the navigation header, with actual UI/UX decisions behind each one instead of placeholder structure. It was enough that the sponsor told us the project was well ahead of where she expected it to be. That had a real consequence too. She'd originally planned on bringing in another round of student developers to keep building it out before handing it off to her own team for final polish. Instead she skipped that step entirely

and went straight into polishing and publishing what we'd built. The version of TRM that's live today is based directly on what my team shipped, and some of the structural decisions I made in the navigation and the pages I owned are still visible on the current site. That outcome traced directly back to the architecture: the routing structure meant every page added after us automatically inherited the correct access rules without anyone having to remember to add a guard clause, and the shared design system helped keep the interface visually consistent across six developers, reducing the need for later UI cleanup before handoff.

Reflection

Looking back, I'd spread myself out more. I came out of this project with a strong grasp of UI/UX and frontend work, but the capstone was really structured as a place to learn, not a place to perform, and looking back I stayed in my strongest area instead of using the time to build up weaker ones. The time on frontend wasn't wasted, that's actually where I found a real interest in UI/UX philosophy that's carried past the project, into how much time I've put into experimenting with and tweaking my own portfolio since. But I think I could have gotten more out of the semester by spending some of it uncomfortable, on backend or database work I wasn't already good at.

The clearest example is the rewrite itself. Our senior developer mentor led the teardown and rebuild of the old codebase into a proper component architecture, and honestly, for good reason, none of us had the git or codebase experience yet to be trusted with something like that. But I never asked to be more involved in it, and I wish I had. Actually restructuring an existing system, instead of building new features on top of a clean one, is a different and pretty useful skill from what I practiced most of the semester, and I let the chance to learn it pass by default instead of by choice.

References

- 1 Frost, B. (2016). *Atomic Design*. Brad Frost. ISBN 978-0-9982966-0-9. Also available at atomicdesign.bradfrost.com.
- 2 OWASP Foundation. *Authorization Cheat Sheet*. OWASP Cheat Sheet Series. cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html.